System and Devices (SYS3)

Lecture Review – Week 7

Dr Pengcheng Liu

COM00018I

Welcome

UNIVERSITY of York

Check-In Code:

434219

Log into checkin.york.ac.uk, find the session and enter the code.

Valid until Thursday, 26 March 2026 11:00

TEF Gold Teaching Excellence Framework

Week 7 Module Overview and Updates

Module Overview and Updates

- **Week 7**:

Lecture: In Week 7, our discussion is focused on some principles of **speculative execution** and **superscalar processors** in Lecture 11 and Lecture 12, respectively.

Lab: We continue working on solutions for Week6's lab tasks: **data hazards**.

Exercises/Quizzes: Solution of Exercise 2 will be released on 27/03/2026 (Week7 Friday) . **Quiz 2** will be discussed in this Lecture Review session.

Exam: A **sample paper** will be released before Easter.

Module Overview and Updates

- **Next Week (Week 8)**:

Lecture: In Week 8, our discussion will be focused on **Virtual Memory** and **Page Replacement** in Lecture 13 and Lecture 14, respectively. And we also have the 3rd tutorial lecture on **Static & Dynamic Scheduling**.

Lab: Register renaming. In previous weeks, we explored register hazards and we noted that creating delays in the pipline to solve these problems is not very efficient, as it wastes CPU clock cycles. An alternative method, register renaming, attempts to solve some of these hazards by another method, which avoids inserting stall cycles. This lab session explores that idea and a solution.

Exercises/Quizzes: Exercise 3 and Quiz 3 will be released on 17/04/2026 (Week 8 Friday). Quiz 3 will be discussed in the Lecture Review session on 23/04/2026 (Week 9 Thursday). Exercise 3 solution will be released on 24/04/2026 (Week 9 Friday) . Exercise 3 will be discussed in the Lecture Review session on 30/04/2026 (Week 10 Thursday).

Questions on Padlet

Padlet Questions

L7 Control Hazards Slide 15- Advance Branch Prediction:

Unsure about what the values in the diagram represent. Can you explain?

The diagram in Slide 15 is a simple illustration of a branch prediction buffer. The first column is the **PC value**, the second column contains a few bits which will tell us the **outcome of the branch**, either taken or not taken. So whenever we have the instructions stored in memory and we have their addresses, we can enter them into the branch prediction buffer, by looking into the lower bits of the PC when a branch is being encountered.

Branch Prediction In Hardware

- **Branch prediction is extremely useful in loops**
- A simple branch prediction can be implemented using a small amount of memory indexed by lower order bits of the address of the branch instruction. (branch prediction buffer)
- One bit stores whether the branch was taken or not.
- The next time the branch instruction is fetched refer this bit.

BPB

2000	1011

Padlet Questions

L7 Control Hazards Slide 15- Advance Branch Prediction:

Unsure about what the values in the diagram represent. Can you explain?

In the BPB, we can use **one bit stored** to tell us if the branch was taken or not.
In this example in the BPB, at PC value 2000, there is a value 1011, which means a branch is being encountered whose address is 2000, the last four outcomes of this branch are: last time it was taken (1), prior to that it was not taken (0), prior to that it was taken (1), and prior to that it was taken (1).
Next time when the branch instruction is fetched we can refer to this second column entry which records the outcome of this branch when it was executed last time, as long as the last few bits of the PC match.

Branch Prediction In Hardware

- **Branch prediction is extremely useful in loops**
- A simple branch prediction can be implemented using a small amount of memory indexed by lower order bits of the address of the branch instruction. (branch prediction buffer)
- One bit stores whether the branch was taken or not.
- The next time the branch instruction is fetched refer this bit.

BPB

2000	1011

Padlet Questions

L7 Control Hazards Slide 16- Advance Branch Prediction:

Correlating and local predictor have a very similar definitions. Please can you clarify the differences?

Advanced Branch Prediction Techniques

- **Basic 2-bit predictor**
  - For each branch: Predict T or NT
  - If the prediction is wrong for two consecutive times, change prediction
- Correlating predictor
  - Multiple 2-bit predictors for each branch
  - One for each possible combination of outcomes of preceding n branches

```
if ( x = = 2)      /* br-1*/
        x = 0;
if ( y = = 2)      /* br-2*/
        y = 0;
if ( x != y)       /* br-3/
            do this
else do that
```

Correlating branch predictors use the **outcomes of last n branches** that to make a prediction, usually when there are dependencies between the branches.
In the example on Slide 16, branch 3 is dependent on branches 1 and 2, so correlating branch predictors will take care of this dependency by using **GLOBAL** information rather than local behavior information. Local predictors only use local histories (outcomes) of the same branch.

Padlet Questions

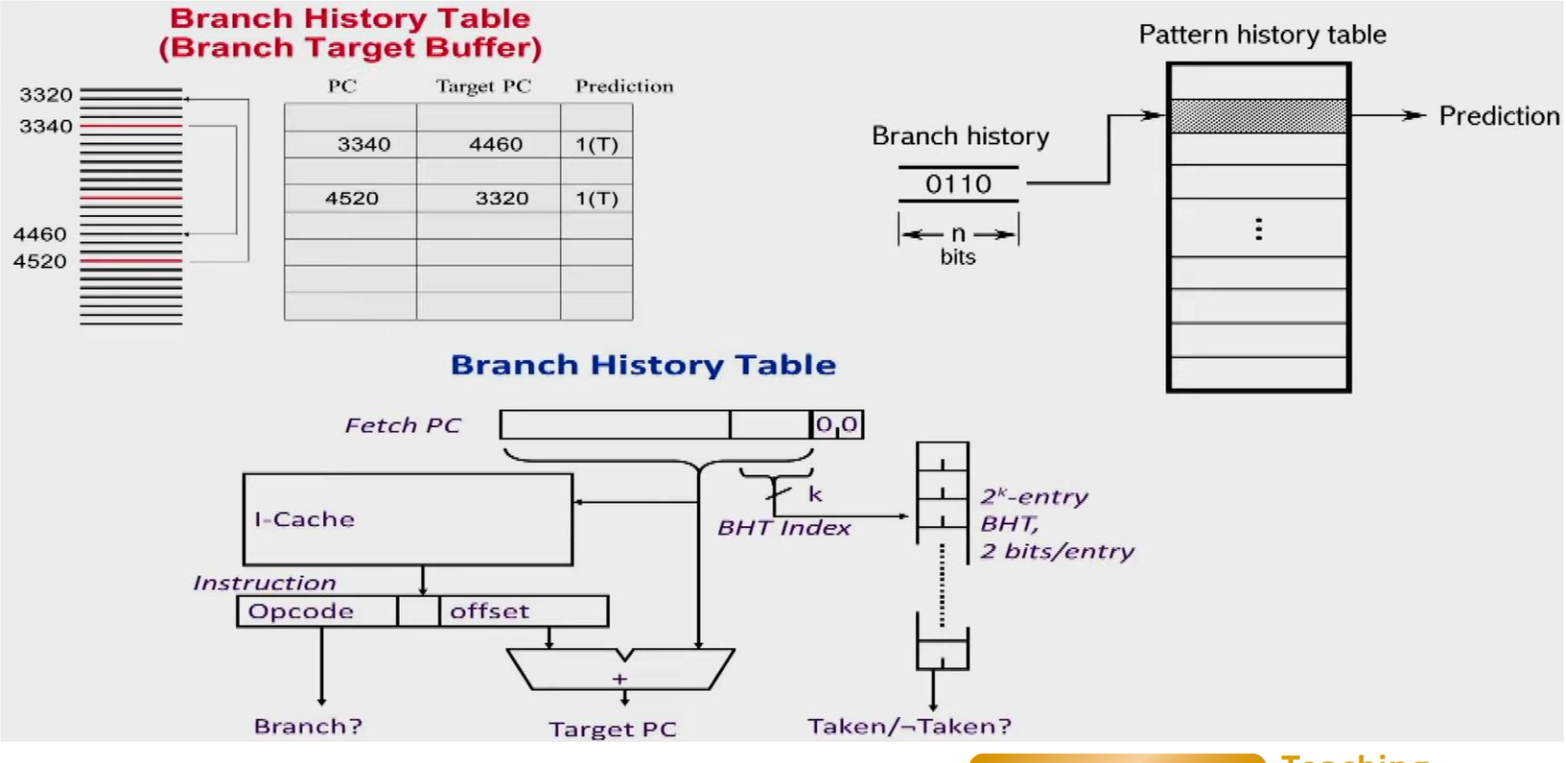
L7 Control Hazards Slide 20- Branch-Target Buffer:

I don't really understand the explanation for slide 20 – Can you explain?

The diagrams on slide 20 are different organisational structures of the branch target buffer. The **diagram on the top left** is a BTB which stores the PC, target PC, and prediction. The diagram **on the top right** is an illustration of a two-level adaptive predictor that remembers the history of the last n occurrences of the branch.

In the example, 0110 represents the history of the branch in the last four occurrences, not taken (0), taken (1), taken (1), and not taken (0). We can have 16 combinations from 0000 to 1111. Based on that we index into the pattern history table, and that will give a prediction of the branch is taken or not taken.

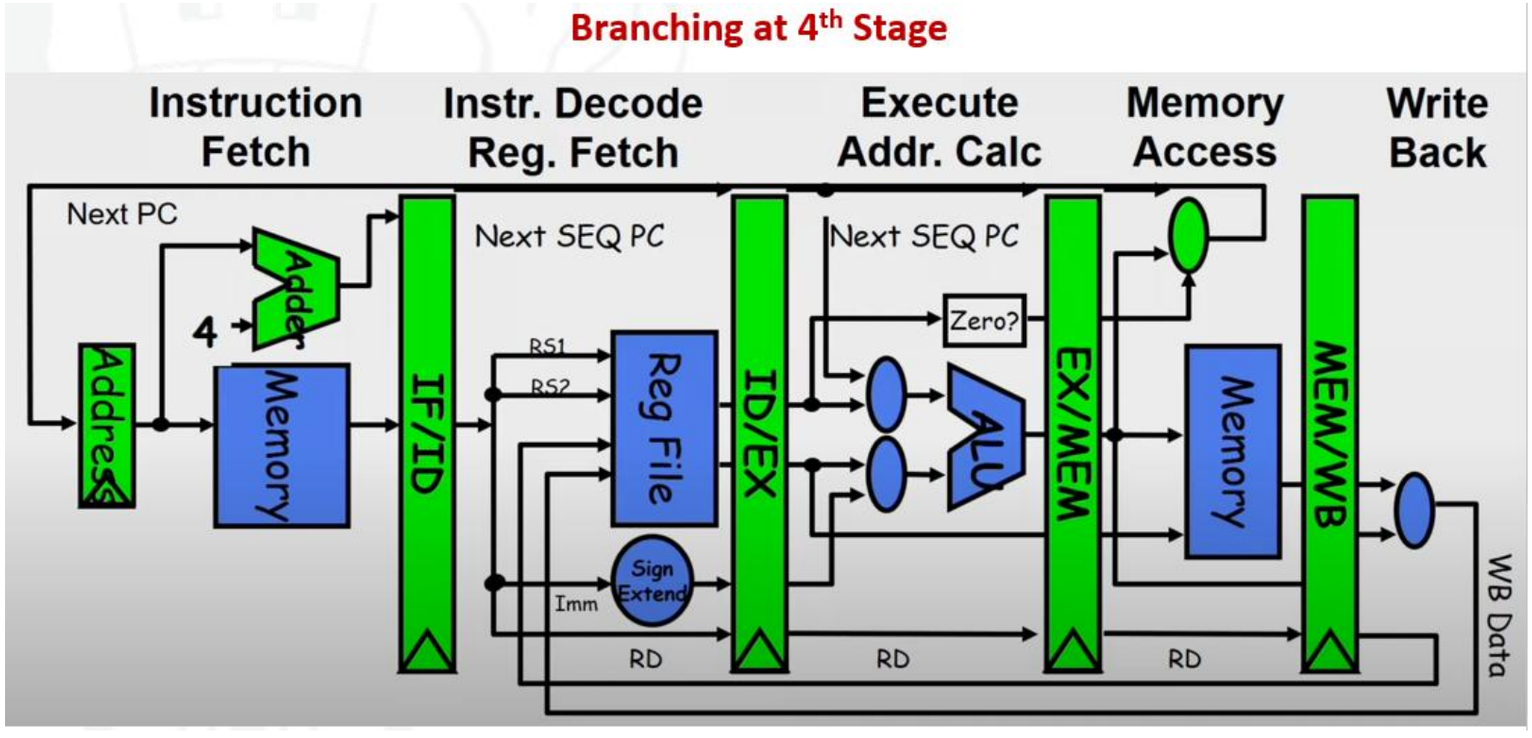
The diagram at the bottom. The workflow is from top to bottom of this diagram. When a PC is fetched, a few bits of its value are used to compute the target PC by doing an addition; and a few least significant bits of its value will be used to index into the branch history table on the right side, and the BHT contains the entry of a state machine to proceed.



Padlet Questions

MEM Phase L5 Instruction Pipeline: *in the memory phase, if it is dealing with the ALU instruction, is memory fetch still carried out? is there a control bit or something to stop a memory fetch happening? also what is the purpose of the green circle in the diagram for the MEM phase?*

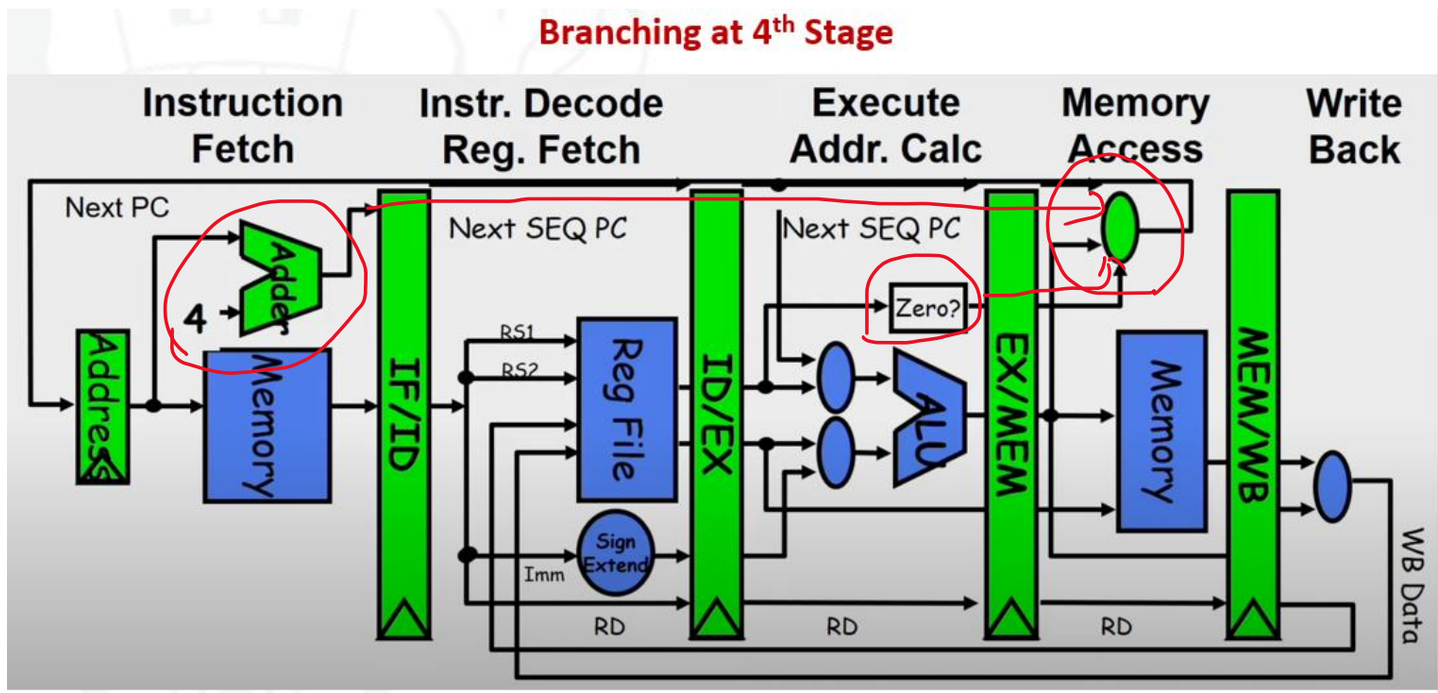
The MEM stage is only used by load and store instructions. If it is an **ALU instruction**, then no operation is carried out in MEM stage, whatever was the result that was available in EX/MEM pipeline register, that is simply by passing in the MEM/WB pipeline register. If it is a **load instruction**, the memory does a read using the effective address computed in the previous cycle (EX stage). If it is a store, then the memory writes the data that is available in the EX/MEM register using the effective address.



Padlet Questions

MEM Phase L5 Instruction Pipeline: *in the memory phase, if it is dealing with the ALU instruction, is memory fetch still carried out? is there a control bit or something to stop a memory fetch happening? also what is the purpose of the green circle in the diagram for the MEM phase?*

The **green circle** in the MEM stage is used for the **branch control logic** in this stage. Normally, branching happens at the fourth stage in the conventional 5-stage pipeline. The normal value of PC + 4 is available at the green circle, and the test zero tells us if we subtract two register values, the result is zero or not, and that is going to tell us whether a branch will happen or not. So branches typically happen from flag registers or flag values, equality checking of two registers. So the **green circle tells the outcome of the branch**: should we continue with the current program counter value, or should we move into or jump into a target location.



Padlet Questions

***L6 Pipeline Hazards– Branches**: What actually is a branch?*

A branch refers to a type of instruction that alters the sequence of program execution by **changing the flow of control** to a different part of the program. Branch instructions are fundamental in computer architecture and are used for implementing **conditional statements** (e.g., if-else constructs) and loops.

Branch instructions specifically **modify the program counter (PC)**, which determines the address of the next instruction to be fetched. Depending on whether the branch condition is true or false, the PC is updated to the address of the target instruction if the branch is taken, or it proceeds to the next sequential instruction if the branch is not taken.

Padlet Questions

***L6 Pipeline Hazards– Branches**: What actually is a branch?*

Branch instructions can be broadly classified into two types:

Unconditional Branch always changes the control flow to a specific target address. Examples include jump (J) instructions in MIPS architecture.

Conditional branches depend on a condition being true or false. The decision to take the branch is based on the evaluation of a specific condition. For example, in MIPS architecture, the branch if equal (**BEQ**) and branch if not equal (**BNE**) instructions are conditional branches that compare the values of two registers and determine whether to take the branch based on the result of the comparison.

Padlet Questions

L7 Control Hazards: *How do you actually predict a branch?*

In a 5-stage RISC pipeline, branch prediction is a crucial aspect of maintaining pipeline efficiency. Here's a simplified overview of how branch prediction works in such pipelines:

IF Stage: In the fetch stage, the processor fetches instructions from memory based on the program counter (PC). If the fetched instruction is a branch instruction, the pipeline needs to predict whether the branch will be taken or not.
ID Stage: In the decode stage, the instruction is decoded, and the control signals for the instruction are generated. For a branch instruction, the target address of the branch is calculated.
EX Stage: In the execute stage, the condition of the branch instruction is evaluated. This evaluation involves comparing the values of the registers or other conditions specified in the branch instruction.

Padlet Questions

L7 Control Hazards: *How do you actually predict a branch?*

In the **MEM stage**, if the branch is a memory-dependent branch, any memory operations associated with the branch are performed. However, in many RISC architectures, branch instructions do not involve memory operations in the memory stage.

In the **WB stage**, the results of ALU operations are written back to registers. For branch instructions, the branch decision (taken or not taken) is finalized.

Padlet Questions

L7 Control Hazards: *How do you actually predict a branch?*

A review on **Branch Prediction Techniques**:
Static Branch Prediction:
Simplest form of branch prediction where the **branch is predicted based on the instruction type or the branch history**.
For example, always predict a forward branch as taken and a backward branch as not taken.
Dynamic Branch Prediction:
Uses **runtime information** to predict the outcome of branches.
Techniques like Branch Target Buffer (**BTB**) and Branch History Table (**BHT**) are commonly used.
Branch Target Buffer stores the target address of previously executed branches along with their outcomes (taken or not taken).
Branch History Table keeps track of the history of branches, allowing the processor to make more accurate predictions based on past behavior.

Padlet Questions

L7 Control Hazards: *How do you actually predict a branch?*

A review on **Branch Prediction Techniques**:

Two-Level Branch Prediction:
Combines the advantages of static and dynamic prediction techniques.
It uses a pattern history table (PHT) to predict branch outcomes based on recent branch history.
PHT indexes a branch history register, allowing the prediction of the branch outcome based on the historical pattern of branch behavior.

Correlation-Based Predictors:
These predictors try to predict the outcome of one branch based on the outcomes of other correlated branches.
They exploit patterns in branch behavior to make more accurate predictions.

Padlet Questions

L7 Control Hazards – Branch taken and not-taken:

Can you explain further Slide 8 and 9, i do not understand the diagrams. Also can you expand further on what is meant by a branch being taken or untaken?

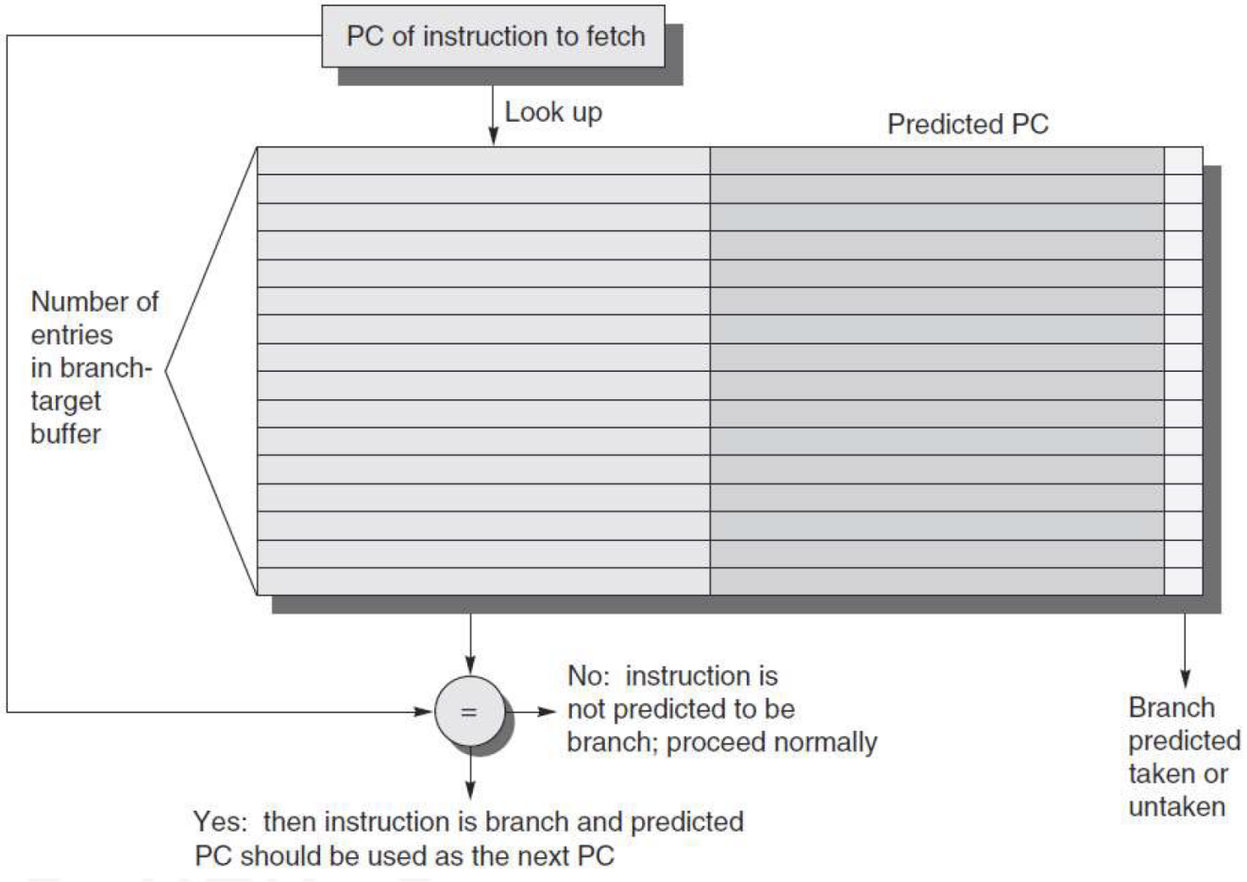
The concept of a branch being "**taken**" or "**not taken**" refers to the outcome of a conditional branch instruction. Conditional branch instructions alter the flow of program execution based on the evaluation of a specific condition. When a conditional branch instruction evaluates the specified condition as true, the processor changes the flow of control by redirecting program execution to a different target address specified by the branch instruction. In other words, if the condition is satisfied, the branch is "**taken**," and the program counter (PC) is updated to the target address of the branch. Conversely, if the condition evaluated by the conditional branch instruction is false, the processor continues executing the next sequential instruction in memory without altering the flow of control. In this case, the branch is considered "**not taken**," and the program counter (PC) simply increments to the address of the next instruction in memory.

Padlet Questions

L7 Control Hazards – BTB, Branch Folding and optimization on branch target buffer to make zero cycle branch:

The Branch Target Buffer is a hardware structure used in modern processors to improve the efficiency of branch instruction execution. It is a **cache-like** component that **stores the target addresses of previously encountered branch instructions** along with their associated branch conditions. The primary function of the BTB is to **predict the target address of upcoming branch instructions** to facilitate faster and more accurate branch resolution.

The BTB caches information about branch instructions that have been encountered during program execution. For each entry in the BTB, it stores the branch instruction's address, its target address, and other relevant metadata.



Padlet Questions

L7 Control Hazards – BTB, Branch Folding and optimization on branch target buffer to make zero cycle branch:

When the processor encounters a branch instruction during instruction fetch, it consults the BTB to **predict the target address of the branch**. Based on the history of branch instructions and their target addresses stored in the BTB, the processor makes a prediction about where the branch will likely jump to.

By predicting the target address of branches, the BTB helps **reduce the delay associated with branch resolution**. Instead of waiting for the branch instruction to complete its execution to determine its target address, the processor can speculatively fetch instructions from the predicted target address, thereby minimizing pipeline stalls and improving overall performance.

Padlet Questions

L7 Control Hazards – BTB, Branch Folding and optimization on branch target buffer to make zero cycle branch:

In this example, the BTB consists of multiple entries, each storing the address of a branch instruction and its corresponding target address.

When the processor encounters a branch instruction during instruction fetch, it searches the BTB for a matching entry based on the branch address. If a match is found, the processor uses the associated target address to predict the outcome of the branch and speculatively fetches instructions from that address.

An example:

```
|--------------------|
| Branch Address 1   | Target Address 1
|--------------------|
| Branch Address 2   | Target Address 2
|--------------------|
| Branch Address 3   | Target Address 3
|--------------------|
| ...                | ...
|--------------------|
| Branch Address N   | Target Address N
|--------------------|
```

Padlet Questions

L7 Control Hazards – Branch Folding and optimization on branch target buffer to make zero cycle branch:

How does the optimization work?

Multiple branch instructions that have similar conditions and occur in close proximity are **merged or folded into a single branch instruction**.

The branch target buffer (BTB) **predicts the target address of the merged branch instruction** based on the historical information stored in the buffer.

When the merged branch instruction is encountered, the predicted target address is immediately fetched and executed without incurring any additional pipeline stalls. This allows the processor to seamlessly continue execution without waiting for the branch resolution, i.e. Zero-Cycle Resolution.

Padlet Questions

L8 Compiler Techniques– Pipeline scheduling: at 10:11 the rearranged instructions for S.D has a different value next to R1 in the rearranged code. Why is this.

In the original code, the SD instruction stores the result F4 back into memory at the same address pointed to by R1. However, in the scheduled code, the 'DADDUI R1, R1, #-8' instruction has been moved up before the ADD.D instruction.

```
Loop: L.D      F0, 0(R1)
      stall
      ADD.D    F4, F0, F2
      stall
      stall
      S.D      F4, 0(R1)
      DADDUI R1, R1, #-8
      stall (assume integer
load latency is 1)
      BNE      R1, R2, Loop
```

Scheduled code:

```
Loop: L.D      F0, 0(R1)
      DADDUI R1, R1, #-8
      ADD.D    F4, F0, F2
      stall
      stall
      S.D      F4, 8(R1)
      BNE      R1, R2, Loop
```

This means that R1 has already been decremented by 8 before the addition operation ADD.D is performed.

Therefore, to store the result F4 at the correct memory address corresponding to the original R1, the index value for SD instruction needs to be updated accordingly. Since R1 has already been decremented by 8, the correct memory address becomes 8(R1) in the scheduled code, making effective compensation.

This adjustment ensures that the data stored by the SD instruction in the scheduled code corresponds to the correct memory location in the original code, despite the reordering of instructions to optimize pipeline scheduling and minimize stalls.

Padlet Questions

***S2 Week 6 L9 - Dynamic scheduling 1**: "Split ID Stage"*

When we talk about splitting the ID stage into two, does this mean that there are 2 clock cycles for ID, or just that there are two branches of logic?

When we talk about splitting the ID stage into two parts, it doesn't necessarily mean there are two dedicated clock cycles for ID. Instead, it refers to **dividing the responsibilities of the ID stage** into two distinct phases: the **issue stage** and the **read operands stage**.

The issue stage handles **decoding** the instruction and **checking** for structural hazards—this is done **in order**. After an instruction is issued, it moves to the read operands stage, where it **waits for its data operands** to become available. At this point, instructions can proceed out of order depending on when their operands are ready.

Padlet Questions

***S2 Week 6 L9 - Dynamic scheduling 1**: "Split ID Stage"*

When we talk about splitting the ID stage into two, does this mean that there are 2 clock cycles for ID, or just that there are two branches of logic?

Whether these stages correspond to specific clock cycles depends on the pipeline's behaviour.

Some instructions may move quickly from issue to execution in back-to-back cycles, while others might stay in the read operands stage longer due to data hazards.

The key idea here is **separating the control logic** into two phases to allow for dynamic scheduling and out-of-order execution, rather than rigidly defining the stages by clock cycle.

Padlet Questions

***S2 Week 6 L9 - Dynamic scheduling 2**: "Register Renaming"*

The left column says "Name dependence with F6". Shouldn't this be a data dependence, since S.D. stores the value of F6 which is written by ADD? If I'm understanding correctly, this is a data dependence, since there is a flow of data between the 2 instructions.

There is a flow of data between the ADD and S.D instructions through F6. However, in this context, "name dependence" refers specifically to situations where two instructions use the same register name (F6), regardless of whether it's a true data dependence or not. This is often called a WAW or WAR hazard. In the case of ADD.D writing to F6 and S.D reading from F6, there's a RAW dependence, which is a true data dependence. But the naming here highlights that the dependence comes from the use of the same register name.

Register Renaming

Name dependence with F6		RAW hazard on T	
DIV.D	F0,F2,F4	DIV.D	F0,F2,F4
ADD.D	F6,F0,F8	ADD.D	S,F0,F8
S.D	F6,0(R1)	S.D	S,0(R1)
SUB.D	F8,F10,F14	SUB.D	T,F10,F14
MUL.D	F6,F10,F8	MUL.D	F6,F10,T

Padlet Questions

***S2 Week 6 L9 - Dynamic scheduling 2**: "Register Renaming"*

Why is the data dependence between DIV and ADD (of F0) not flagged?

The dependence between DIV.D and ADD.D through F0 is a data dependence (RAW hazard), but the slide focuses on the dependencies involving F6 to demonstrate how register renaming resolves name dependencies and hazards. The F0 dependence would still need to be managed, but it's not the emphasis here.

Register Renaming

Name dependence with F6		RAW hazard on T	
DIV.D	F0,F2,F4	DIV.D	F0,F2,F4
ADD.D	F6,F0,F8	ADD.D	S,F0,F8
S.D	F6,0(R1)	S.D	S,0(R1)
SUB.D	F8,F10,F14	SUB.D	T,F10,F14
MUL.D	F6,F10,F8	MUL.D	F6,F10,T

Padlet Questions

***S2 Week 6 L9 - Dynamic scheduling 2**: "Register Renaming"*

The content of the video says that renaming F6 as S removes a WAW hazard. Why / how is this, please?

Both ADD.D and MUL.D write to F6 in the original code, which creates a **WAW hazard**: they both try to write to the same register. Renaming F6 as S in the ADD.D instruction (and its consumer S.D) removes this conflict because they **no longer compete for F6**.
Now, MUL.D is the only instruction writing to F6, and ADD.D writes to S, eliminating the WAW hazard.

Register Renaming

Name dependence with F6		RAW hazard on T	
DIV.D	F0,F2,F4	DIV.D	F0,F2,F4
ADD.D	F6,F0,F8	ADD.D	S,F0,F8
S.D	F6,0(R1)	S.D	S,0(R1)
SUB.D	F8,F10,F14	SUB.D	T,F10,F14
MUL.D	F6,F10,F8	MUL.D	F6,F10,T

Padlet Questions

***S2 Week 6 L9 - Dynamic scheduling 2**: "Register Renaming"*

Is there a reason why the label of one column focuses on 'dependence' while the label of the other focuses on 'hazard'?

The left column shows "name dependence with F6," **highlighting where the same register name is reused**, which could potentially cause hazards. The right column shows the "RAW hazard on T" after renaming, illustrating how the true data dependences are managed with renamed registers, and what hazards remain post-renaming.

Register Renaming

Name dependence with F6		RAW hazard on T	
DIV.D	F0,F2,F4	DIV.D	F0,F2,F4
ADD.D	F6,F0,F8	ADD.D	S,F0,F8
S.D	F6,0(R1)	S.D	S,0(R1)
SUB.D	F8,F10,F14	SUB.D	T,F10,F14
MUL.D	F6,F10,F8	MUL.D	F6,F10,T

Padlet Questions

***S2 Week 6 L9 - Dynamic scheduling 2**: "Register Renaming"*

What this renaming these registers actually achieving? What is the benefit of renaming the registers to 'S' and 'T'? (I presume this is covered later?)

Renaming registers to S and T removes unnecessary constraints. It **eliminates WAW and WAR hazards** by making sure each instruction writes to a unique temporary location rather than sharing the same register name. This allows instructions to execute more **independently** and **out of order**, improving instruction-level parallelism and overall performance.
In later lectures, we will explore how Tomasulo's algorithm and dynamic scheduling leverage this to enable more efficient execution.

Register Renaming

Name dependence with F6		RAW hazard on T	
DIV.D	F0,F2,F4	DIV.D	F0,F2,F4
ADD.D	F6,F0,F8	ADD.D	S,F0,F8
S.D	F6,0(R1)	S.D	S,0(R1)
SUB.D	F8,F10,F14	SUB.D	T,F10,F14
MUL.D	F6,F10,F8	MUL.D	F6,F10,T

Padlet Questions

***S2 Week 6 L9 - Dynamic scheduling 2**: Are Reservation Stations FIFO Queues? I noted while listening to the lecture that individual RS were FIFO queues but then in the textbook it talks about picking the next instruction to execute arbitrarily so I might have misunderstood.*
Textbook: "... if more than one instruction is ready for a single functional unit, the unit will have to choose among them. For the floating point reservation-stations, this choice may be made arbitrarily; loads and stores, however, present an additional complication."

The instruction queue in Tomasulo's algorithm is handled **in FIFO order**. Instructions are fetched from this queue and issued one by one in program order.
However, once an instruction has been issued to a reservation station, things change. While the reservation stations were described in the lecture as "queues", they don't operate strictly as FIFO queues. Instead, they hold instructions **waiting to execute until their operands are ready and the functional unit becomes available.**

Padlet Questions

***S2 Week 6 L9 - Dynamic scheduling 2**: Are Reservation Stations FIFO Queues? I noted while listening to the lecture that individual RS were FIFO queues but then in the textbook it talks about picking the next instruction to execute arbitrarily so I might have misunderstood.*
Textbook: "... if more than one instruction is ready for a single functional unit, the unit will have to choose among them. For the floating point reservation-stations, this choice may be made arbitrarily; loads and stores, however, present an additional complication."

When multiple instructions are waiting in reservation stations, the **execution order can be out-of-order**. As soon as operands become available and the functional unit is free, an instruction can proceed, regardless of its original issue order. So, while **issue happens in FIFO order**, execution from the reservation stations does not follow FIFO rules.
The textbook's statement reflects: if more than one instruction is ready in a reservation station, the hardware may pick any ready instruction to proceed. It's not bound to FIFO ordering at this stage.

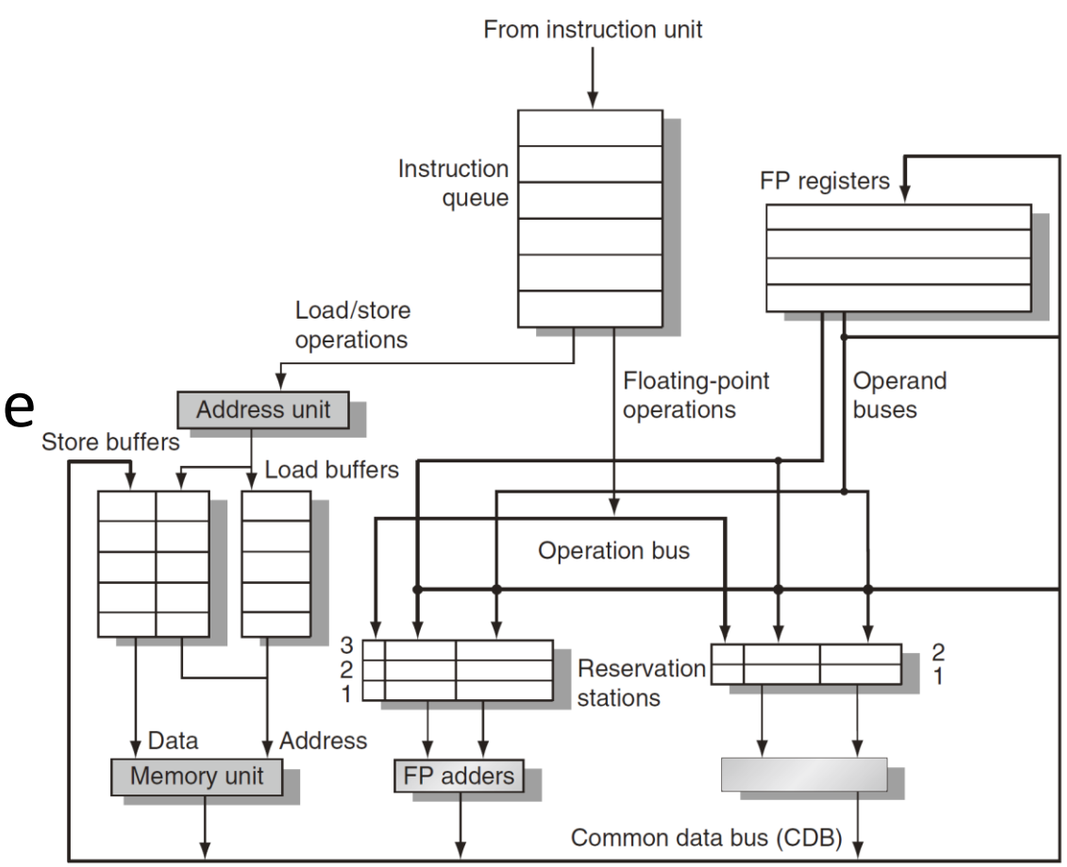
Padlet Questions

***S2 Week 6 L9 - Dynamic scheduling 2**: Reservation Stations Map*
What is the difference between the 2 groups of Reservation Stations? There look to be one group of 3 and one group of 2. What is the difference?

In this diagram, RS are categorized into two groups based on the type of functional unit they serve.

First Group (3-Entry Reservation Station - Floating Point Adders)

- This reservation station serves floating-point addition and subtraction operations.
- Instructions that involve FP adders are fed into this queue after being picked from the instruction queue.
- If the operands are available, they are retrieved from the registers and placed in the reservation station.
- Once the functional unit (adder) is free, the operation is executed, and results are sent back via the CDB.



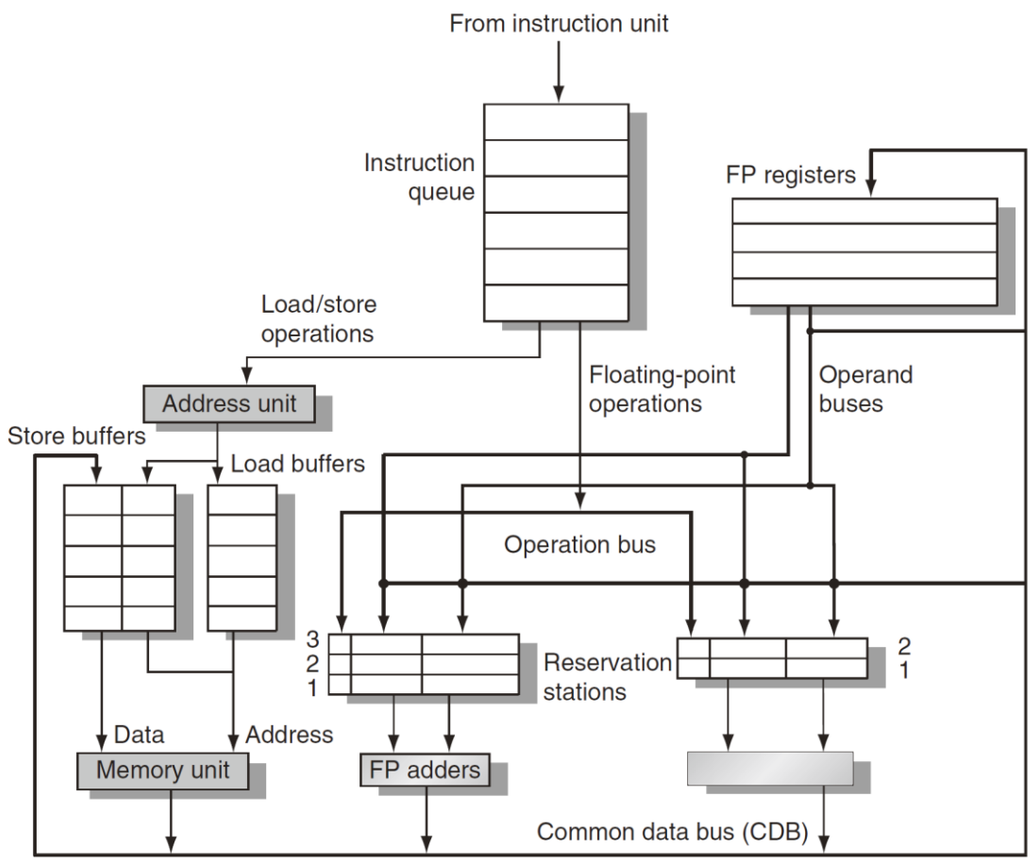
Padlet Questions

***S2 Week 6 L9 - Dynamic scheduling 2**: Reservation Stations Map*
What is the difference between the 2 groups of Reservation Stations? There look to be one group of 3 and one group of 2. What is the difference?

Second Group (2-Entry Reservation Station - Floating Point Multipliers)

- This reservation station handles floating-point multiplication operations.
- Instructions requiring multiplication enter this queue and wait until their operands become available.
- Similar to the first group, once the functional unit is free, the values are fetched, and the operation is executed.

The number of entries reflects the **expected frequency** and **demand for each type of operation**.
CDB ensures that results are broadcasted to waiting instructions in the reservation stations and updated in registers.



SYS3 Exercises/Quizzes Discussions

Quiz 2

Quiz 2

Question 1

1 point

The contention for the usage of a hardware device is called _____

(A) Structural hazard

(B) Stalk

(C) Deadlock

(D) None of the mentioned

A (A) Correct answer

B (B)

C (C)

D (D)

Quiz 2

Question 2

1 point

The situation wherein the data of operands are not available is called _____

(A) Data hazard

(B) Stock

(C) Deadlock

(D) Structural hazard

(A) (A) Correct answer

(B) (B)

(C) (C)

(D) (D)

Data hazards are generally caused when the data is not ready on the destination side.

Quiz 2

Question 3

1 point

Any condition that causes a processor to stall is called as _____

(A) Hazard

(B) Page fault

(C) System error

(D) None of the mentioned

Ⓐ (A) Correct answer

Ⓑ (B)

Ⓒ (C)

Ⓓ (D)

A hazard causes a delay in the execution process of the processor.

Quiz 2

UNIVERSITY of York

Question 4

1 point

The time lost due to the branch instruction is often referred to as _____

(A) Latency

(B) Delay

(C) Branch penalty

(D) None of the mentioned

A (A)

B (B)

C (C) Correct answer

D (D)

This time also retards the performance speed of the processor.

Quiz 2

Question 5

1 point

The stalling of the processor due to the unavailability of the instructions is called as _____

(A) Control hazard

(B) structural hazard

(C) Input hazard

(D) None of the mentioned

- (A) Correct answer
- (B)
- (C)
- (D)

The control hazard also called as instruction hazard is usually caused by a cache miss.

Quiz 2 Q5

The stalling of the processor due to the unavailability of the instructions is called as control hazard.

It arises from the **uncertainty about the correct control flow of the program**. This typically happens in a pipelined processor when the instruction fetch stage depends on the outcome of a branch or jump instruction, but that outcome is not yet determined.

- Control flow dependency. The hazard occurs due to uncertainty in determining the next instruction to be executed. This happens when the processor encounters a control instruction (such as a branch, jump, or function call) whose target address is not immediately available.
- Pipeline stalling. Since the processor does not know which instruction to fetch next, it may have to stall (pause instruction execution) until the correct instruction address is determined. This delay is referred to as a control hazard.
- Branch instructions as a cause. Control hazards are commonly associated with branch instructions (like conditional jumps), where the decision to take a branch or continue sequential execution is made later in the pipeline, leading to a stall if the instruction fetch stage cannot proceed.
- Performance impact. Since pipelining relies on continuously fetching and executing instructions, a control hazard disrupts this flow, reducing the efficiency of the pipeline.

Quiz 2

Question 6

1 point

The pipeline bubbling is a method used to prevent data hazard and structural hazards.

True — *Correct answer*

False

The periods of time when the unit is idle is called a Bubble.

Quiz 2

Question 7

1 point

The pipelining process is also called as _____

(A) Superscalar operation

(B) Assembly line operation

(C) Von Neumann cycle

(D) None of the mentioned

- (A) (A)
- (B) (B) Correct answer
- (C) (C)
- (D) (D)

It is called so because it performs its operation at the assembly level.

Quiz 2

UNIVERSITY of York

Question 8

1 point

Which of the following is the disadvantage of Pipelining?
(A) cycle time of the processor is reduced.
(B) The design of a pipelined processor is complex and costly to manufacture.
(C) The instruction latency is more.
(D) Both B and C

A (A)

B (B)

C (C)

D (D) Correct answer

Quiz 2 Q8

Cycle time of the processor is reduced

- Pipelining increases instruction throughput by allowing multiple instructions to be processed simultaneously at different stages.
- As a result, the processor can complete more instructions per unit time, effectively reducing the average instruction execution time.
- This does not mean the actual cycle time (clock period) of the processor is reduced; rather, the time per instruction is reduced due to overlapping execution.

Quiz 2

UNIVERSITY of York

Question 9

1 point

Which of the following is an advantage of pipelining?
(A) Instruction throughput increases.
(B) Faster ALU can be designed when pipelining is used.
(C) Pipelining increases the overall performance of the CPU.
(D) All of the above

A (A)

B (B)

C (C)

D (D) Correct answer

Quiz 2

Question 10

1 point

For a pipelined CPU with a single ALU, consider the following situations
I. The j+1 –st instruction uses the result of the j-th instruction as an operand
II. The execution of a conditional jump instruction
III. The j-th and j+1st instructions require the ALU at the same time.
Which of the above can cause a hazard:
(A) I and II only.
(B) II and III only.
(C) III only.
(D) All the three.

- A (A)
- B (B)
- C (C)
- D (D) Correct answer

Quiz 2 Q10

I. The j+1 –st instruction uses the result of the j-th instruction as an operand

- This is a data hazard (specifically, a read-after-write (RAW) hazard).
- The *j* + 1 j+1-st instruction requires the result of the *j* j-th instruction, but the result is not available immediately. Solution:
- Forwarding (data forwarding) or pipeline stalling can be used to handle this.

Quiz 2 Q10

II. The execution of a conditional jump instruction

- This is a control hazard because the next instruction's execution depends on whether the branch is taken or not.
- Until the branch decision is resolved, the pipeline does not know which instruction to fetch next, leading to a stall.

Quiz 2 Q10

III. The j-th and j+1-st instructions require the ALU at the same time

- This is a structural hazard because there is only a single ALU, and both instructions cannot execute their ALU operation simultaneously.
- Structural hazards occur when hardware resources are insufficient to support all instructions in the pipeline.

Quiz 2

Question 11

1 point

Which of the following are NOT true in a pipelined processor?
I. Forwarding (aka Bypassing) can handle all RAW hazards
II. Register renaming can eliminate all register carried WAR hazards
III. Control hazard penalties can be eliminated by dynamic branch prediction
(A) I and II only
(B) I and III only
(C) II and III only
(D) I, II and III

- A (A)
- B (B) Correct answer
- C (C)
- D (D)

Quiz 2 Q11

I. Forwarding (aka Bypassing) can handle all RAW hazards

False

- Forwarding (bypassing) allows the result of an instruction to be passed directly to a dependent instruction without waiting for it to be written to a register.

- However, forwarding cannot handle all RAW hazards.

Example: If the instruction causing the hazard is a load (LW) instruction, forwarding is not sufficient because the data is only available after memory access (EX stage cannot forward from MEM stage). This leads to a load-use stall.

- Thus, stalls may still be needed in some cases.

Quiz 2 Q11

II. Register renaming can eliminate all register carried WAR hazards

True

- Write-after-Read (WAR) hazards occur when an instruction writes to a register before an earlier instruction has finished reading from it.
- Register renaming eliminates WAR hazards because it ensures that different instructions do not interfere with each other's register use by assigning unique physical registers.

Quiz 2 Q11

III. Control hazard penalties can be eliminated by dynamic branch prediction

False

- Dynamic branch prediction reduces control hazard penalties but does not eliminate them completely.
- Even with a highly accurate branch predictor, there are still cases of branch mispredictions, which require pipeline flushing and result in penalties.
- Thus, control hazards cannot be completely eliminated.

Quiz 2

UNIVERSITY of York

Question 12

1 point

Register renaming is done in pipelined processors
(A) as an alternative to register allocation at compile time
(B) for efficient access to function parameters and local variables
(C) to handle certain kinds of hazards
(D) as part of address translation

A (A)

B (B)

C (C) Correct answer

D (D)

Register renaming is done to eliminate WAR (Write after Read) and WAW (Write after Write) dependency between instructions which could have caused pipeline stalls. Hence (C) is the answer.

Quiz 2

UNIVERSITY of York

Question 13

1 point

The _____ plays a very vital role in the case of superscalar processors.

- (A) Compilers — Correct answer
- (B) Motherboard
- (C) Memory
- (D) Peripherals

The compilers are programmed to arrange the instructions to get more throughput.

Quiz 2

UNIVERSITY of York

Question 14

1 point

The primary goal of branch prediction is to:

- ○ Reduce memory latency
- ○ Avoid structural hazards
- ● Guess the outcome of branches to reduce pipeline stalls — Correct answer
- ○ Increase register capacity

Branch prediction reduces control hazard penalties by predicting the next instruction path.

Quiz 2

UNIVERSITY of York

Question 15

1 point

Which of the following describes static branch prediction?

- Prediction based on runtime behaviour
- Prediction based on hardware history tables
- Prediction determined before program execution — Correct answer
- Prediction using a branch target buffer

Static prediction uses fixed rules, such as "always taken" or "always not taken".

Quiz 2

UNIVERSITY of York

Question 16

1 point

Loop unrolling is used to:

- Reduce instruction size
- Increase instruction-level parallelism — Correct answer
- Reduce memory access latency
- Increase branch frequency

Unrolling exposes more independent instructions, enabling greater ILP.

Quiz 2

UNIVERSITY of York

Question 17

1 point

Dynamic scheduling allows the processor to:

- ○ Execute instructions strictly in program order
- ● Execute instructions out of order when operands are ready — Correct answer
- ○ Eliminate all hazards
- ○ Reduce instruction count

Dynamic scheduling improves performance by allowing out-of-order execution.

Quiz 2

UNIVERSITY of York

Question 18

1 point

A key improvement of Tomasulo's algorithm over scoreboarding is:

- ○ Use of branch prediction
- ● Register renaming — Correct answer
- ○ Static instruction scheduling
- ○ Reduced instruction fetch time

Register renaming removes WAR and WAW hazards.

Quiz 2

UNIVERSITY of York

Question 19

1 point

In Tomasulo's algorithm, reservation stations:

- Store completed results
- Hold instructions waiting for operands and functional units — Correct answer
- Replace registers
- Execute instructions

Reservation stations buffer instructions until operands and functional units are available.

Quiz 2

UNIVERSITY of York

Question 20

1 point

The Common Data Bus in Tomasulo's algorithm is used to:

- Fetch instructions
- Broadcast computed results to waiting instructions — Correct answer
- Transfer instructions to memory
- Predict branches

The CDB allows results to be broadcast to all reservation stations waiting for them.

Quiz 2

UNIVERSITY of York

Question 21

1 point ...

Speculative execution allows a processor to:

- ○ Execute instructions after confirming branch outcomes
- ● Execute instructions before branch outcomes are known — Correct answer
- ○ Eliminate branch instructions
- ○ Remove hazards completely

Instructions are executed speculatively, assuming a predicted branch direction.

Quiz 2

UNIVERSITY of York

Question 22

1 point

The reorder buffer ensures:

- ○ Instructions execute in order
- ● Results are committed in program order — Correct answer
- ○ Memory accesses occur first
- ○ Branches are eliminated

ROB maintains precise exceptions and correct program order for commits.

Quiz 2

UNIVERSITY of York

Question 23

1 point

Which factor most limits superscalar performance?

- ○ Memory size
- ● Instruction-level parallelism in the program — Correct answer
- ○ Clock speed
- ○ Number of registers

Superscalar processors rely on available ILP in the instruction stream.

Quiz 2

UNIVERSITY of York

Question 24

1 point

Forwarding cannot fully resolve hazards in which case?

- ALU-ALU dependency
- Load-use dependency — Correct answer
- Store-store dependency
- Independent instructions

Load data is only available after MEM stage, so a stall is still required.

Quiz 2

UNIVERSITY of York

Question 25

1 point

As pipeline depth increases, branch misprediction penalty:

- Decreases
- Remains constant
- Increases — Correct answer
- Becomes zero

More stages → more instructions must be flushed → higher penalty.

Quiz 2

UNIVERSITY of York

Question 26

1 point

In Tomasulo's algorithm, instructions execute:

- Strictly in program order
- In reverse order
- Only after all instructions are issued
- When operands and functional units are available — Correct answer

Execution is data-driven, not program-order driven.

Quiz 2

UNIVERSITY of York

Question 27

1 point

A BTB stores:

- Data values
- Instruction opcodes
- Predicted branch targets — Correct answer
- Register contents

BTB allows immediate fetching of the predicted target address.

Quiz 2

UNIVERSITY of York

Question 28

1 point

In a deep pipeline, branch misprediction penalty is high because:

- Instructions execute slower
- More instructions must be flushed — Correct answer
- Memory latency increases
- ALU operations are delayed

More pipeline stages → more in-flight instructions → larger flush cost

Quiz 2

UNIVERSITY of York

Question 29

1 point

A processor can issue 4 instructions per cycle, but the program only has 2 independent instructions at any time. What is the effective issue rate?

- 1
- 2 — Correct answer
- 3
- 4

Performance is limited by available ILP, not hardware width.

Quiz 2

UNIVERSITY of York

Question 30

1 point

Which statement is correct?

- Forwarding removes WAR hazards
- Renaming removes RAW hazards
- Forwarding helps RAW hazards; renaming removes WAR/WAW — Correct answer
- Both solve control hazards

Forwarding → resolves true dependencies (RAW); Renaming → removes false dependencies (WAR/WAW)

Discussion Forum

Discussion Forum - Key Themes for Discussion

UNIVERSITY of York

1. Pipeline Hazards & Mitigation Strategies

Types of hazards: data, control, and structural.

Solutions such as forwarding, stall cycles, and hazard detection mechanisms.

2. Pipeline Depth & Performance Trade-offs

Impact of deeper pipelines on speed, efficiency, and diminishing returns.

The role of loop unrolling and instruction scheduling in optimizing performance.

Discussion Forum - Key Themes for Discussion

UNIVERSITY of York

3. Compiler Optimisation Across Architectures

Differences in RISC (e.g., ARM) vs. CISC (e.g., x86) optimizations.

How instruction scheduling, vectorization, and memory management improve execution efficiency.

Discussion Forum - Pipeline & Compilers

Pick one or more of the following questions, discuss with your small group and add comments to the Padlet discussion board:

1. What are pipeline hazards in RISC architectures?
2. Can WAR and WAW happen in in-order 5-stage pipeline?
3. How to resolve structural hazards? What are the cons and pros of these methods?
4. How does forwarding (or bypassing) mitigate data hazards in a RISC pipeline?

Discussion Forum - Pipeline & Compilers

Pick one or more of the following questions, discuss with your small group and add comments to the Padlet discussion board:

5. Does increasing the depth of pipelining always increase performance?

6. How does loop unrolling contribute to performance improvement? What are the trade-offs?

7. How does instruction scheduling work, and why is it important for modern CPU architectures?

8. How do compilers optimise code for different hardware architectures (e.g., ARM vs. x86)?

Discussion Forum - Approach

Small Group Discussions

– Form a small group with someone nearby, or feel free to move around and join your friends

– Pick one or more applications

– 15 mins group discussion

– Add comments/summary as a group to the Padlet discussion board

https://padlet.com/liupengcheng83/week-7-discussion-forum-u2hu2h14429kh42m

Discussion Forum - Pipeline Hazards

UNIVERSITY of York

1. What are pipeline hazards in RISC architectures?

Pipeline hazards in RISC architectures are situations where the smooth flow of instructions through the pipeline is disrupted, leading to performance degradation or incorrect program behavior.

These hazards can occur due to various factors and can be broadly categorised into three main types:

Data Hazards, Structural Hazards and Control Hazards.

Discussion Forum - Pipeline Hazards

UNIVERSITY of York

1. What are pipeline hazards in RISC architectures?

Data Hazards:

Data hazards occur when an instruction depends on the result of a previous instruction that has not yet completed its execution. This dependency can arise due to read-after-write (RAW), write-after-read (WAR), or write-after-write (WAW) dependencies.

Examples include:

RAW hazard: When an instruction needs a data item that a previous instruction has not yet written back.

WAR hazard: When an instruction writes a result that is needed by a previous instruction before it completes its execution.

WAW hazard: When two instructions write to the same destination, and the order of writing affects the result.

Discussion Forum - Pipeline Hazards

1. What are pipeline hazards in RISC architectures?

Structural Hazards:

Structural hazards occur when the hardware resources required by multiple instructions are not available simultaneously. This may include conflicts in accessing the same functional unit, memory port, or other hardware resource.

Examples include:

A situation where two instructions require access to the same functional unit (e.g., ALU, multiplier) at the same time.

Concurrent access conflicts to the data or instruction cache.

Discussion Forum - Pipeline Hazards

UNIVERSITY of York

1. What are pipeline hazards in RISC architectures?

Control Hazards:

Control hazards arise due to changes in the program flow, such as branches, jumps, or other control transfer instructions. These hazards can cause pipeline bubbles (stalls) when the correct target address is not immediately known.

Examples include:

Branches that change the program counter (PC) may cause the pipeline to speculate on the branch outcome, leading to incorrect predictions and wasted cycles.

Conditional branches that depend on the results of previous instructions may introduce delays in the pipeline until the branch outcome is determined.

Discussion Forum - Pipeline Hazards

UNIVERSITY of York

2. Can WAR and WAW happen in in-order 5-stage pipeline?

A WAR hazard arises when an instruction writes to a register before a previous instruction reads from the same register. However, in a 5-stage pipeline, all instructions take 5 stages, a write can occur in stage 5, while the earliest a read can occur is in the Decode stage. Therefore, by the time an instruction can write to a register, any previous instruction that reads from the same register will have already done so, thus avoiding a WAR hazard.

WAR hazards can happen in **out-of-order pipelines**. But in an in-order 5-stage pipeline, WAW hazards typically do not occur because the WB stage is the only stage in which the register file is updated. Therefore, two instructions that write to the same register cannot both write to the register in the same cycle. If two such instructions are executed out-of-order, then the later instruction will write to the register after the earlier instruction has already written to it, causing a WAW hazard. However, in an in-order pipeline, instructions are always executed in program order, so WAW hazards do not occur.

Discussion Forum - Pipeline Hazards

UNIVERSITY of York

2. Can WAR and WAW happen in in-order 5-stage pipeline?

In an in-order 5-stage pipeline, WAR and WAW hazards are generally prevented due to the nature of the pipeline design. In an in-order pipeline, **instructions progress through the pipeline stages in the order** in which they were fetched, and the pipeline ensures that write operations do not complete out of order.

In an in-order pipeline, instructions move through the pipeline stages in **a strict sequential order**. Each stage completes its operation before the next stage begins for the next instruction. The write-back (WB) stage, where the result of an operation is written back to the register file, follows the execute (EX) and memory access (MEM) stages. This ensures a natural order of execution.

Discussion Forum - Pipeline Hazards

UNIVERSITY of York

2. Can WAR and WAW happen in in-order 5-stage pipeline?

It's important to note that while in-order pipelines simplify the handling of data dependencies, they may **not fully exploit instruction-level parallelism** compared to out-of-order pipelines. Out-of-order pipelines, by contrast, allow instructions to execute out of order and may **require additional mechanisms** (e.g., scoreboarding, reservation stations) to handle hazards and maintain correctness.

Discussion Forum - Pipeline Hazards

3. How to resolve structural hazards? What are the cons and pros of these methods?

Structural hazards in computer architecture refer to conflicts that arise when multiple instructions need the same hardware resource simultaneously. These conflicts can hinder the pipeline's ability to execute instructions concurrently. Some Methods:

Resource Duplication:

Pros: Simple and straightforward solution. Eliminates conflicts by duplicating the hardware resource.

Cons: Increases hardware complexity and cost. May not be feasible for all resources due to space and power constraints.

Discussion Forum - Pipeline Hazards

3. How to resolve structural hazards? What are the cons and pros of these methods?

Resource Partitioning:

Pros: Divides the resource into multiple partitions, allowing parallel use. Reduces conflicts without fully duplicating the resource.

Cons: Adds complexity to resource management and scheduling. Limited improvement if the resource contention is still high.

Pipelining:

Pros: Breaks down the execution of an instruction into stages, allowing multiple instructions to overlap. Reduces the impact of structural hazards by dividing the resource usage across stages.

Cons: May introduce pipeline stalls and hazards, such as data hazards or control hazards. Requires careful instruction scheduling and handling of hazards.

Discussion Forum - Pipeline Hazards

3. How to resolve structural hazards? What are the cons and pros of these methods?

Instruction Scheduling:

Pros: Reorders instructions to avoid conflicts and optimize resource utilization. Minimizes the impact of structural hazards by careful instruction sequencing.

Cons: Requires sophisticated algorithms for optimal scheduling. May introduce additional dependencies or complicate code generation.

Dynamic Scheduling:

Pros: Allows instructions to be scheduled dynamically at runtime based on availability of resources. Adapts to changing program behavior.

Cons: Increases complexity in the design of the instruction scheduler. May introduce overhead due to scheduling decisions at runtime.

Discussion Forum - Pipeline Hazards

3. How to resolve structural hazards? What are the cons and pros of these methods?

Hardware Interlocking:

Pros: Uses hardware mechanisms to prevent conflicting accesses to resources. Ensures that only one instruction at a time can use a specific resource.

Cons: May introduce delays and reduce throughput if contention is high. Can limit parallelism and concurrency.

Discussion Forum - Pipeline Hazards

UNIVERSITY of York

4. How does forwarding (or bypassing) mitigate data hazards in a RISC pipeline?

Forwarding, also known as bypassing, allows data to be passed directly from one stage of the pipeline to another, bypassing intermediate stages. This helps mitigate data hazards by providing the necessary operands to instructions without waiting for them to be written back to the register file.

The pipeline **detects data hazards** when an instruction in the execution stage requires data that has not yet been produced by a preceding instruction in an earlier stage of the pipeline.

When a data hazard is detected, the pipeline checks if the required data is available in a later pipeline stage of the same instruction. If the data is available, it can be **forwarded directly to the stage** where it is needed, bypassing the intermediate stages.

The **bypassing mechanism** connects the output of certain pipeline stages (such as the execute stage) to the input of earlier stages (such as the decode or execute stage) to provide the required data directly to instructions that need it.

Discussion Forum - Pipeline Hazards

UNIVERSITY of York

4. How does forwarding (or bypassing) mitigate data hazards in a RISC pipeline?

By forwarding data directly, the pipeline avoids the need to wait for the data to be written back to the register file and then read again in subsequent stages. This **minimizes pipeline stalls** and ensures that instructions can proceed through the pipeline without unnecessary delays.

RISC pipelines often implement **multiple forwarding paths** to handle different types of data hazards efficiently. For example, forwarding paths may be provided for data produced by arithmetic operations, memory operations, or register-to-register transfers.

In some cases, forwarding may not be possible due to structural dependencies or hazards that cannot be resolved through bypassing. In such situations, the **pipeline may introduce interlocks or stalls** to prevent incorrect results or hazards from propagating through the pipeline.

Discussion Forum - Pipeline Hazards

UNIVERSITY of York

5. Does increasing the depth of pipelining always increase performance?

Increasing the depth of pipelining does not always lead to a straightforward improvement in performance. While deeper pipelines can potentially increase the overall throughput and allow for higher clock frequencies, there are several trade-offs and challenges associated with deeper pipelines that may impact performance:

Deeper pipelines can exacerbate **the impact of pipeline stalls**. Longer pipelines mean that when a stall occurs (e.g., due to a cache miss or a branch misprediction), it takes more clock cycles for the pipeline to recover, leading to a performance penalty.

Deeper pipelines increase the number of stages an instruction must pass through before its result is produced. This results in **increased instruction latency**, meaning it takes longer for an instruction to complete its execution.

Discussion Forum - Pipeline Hazards

5. Does increasing the depth of pipelining always increase performance?

Deeper pipelines can introduce **increased contention for critical resources**, such as register files and functional units. This contention can lead to delays and reduce overall performance.

Managing control hazards, such as branch mispredictions, becomes more challenging in deeper pipelines. The **increased complexity in control hazard handling** can impact overall performance.

Deeper pipelines can lead to **longer clock cycle times**, potentially **limiting the achievable clock frequency**. While deeper pipelines allow for more stages to be completed in each cycle, the overall clock cycle time needs to be considered for performance evaluation.

Discussion Forum - Pipeline Hazards

UNIVERSITY of York

5. Does increasing the depth of pipelining always increase performance?

The effectiveness of deeper pipelines is often **dependent on compiler optimizations**. The compiler needs to schedule instructions efficiently to make optimal use of the pipeline stages. Poorly optimized code may not fully exploit the benefits of a deep pipeline.

Deeper pipelines may result in **increased dynamic power consumption** due to the higher number of gates and stages toggling in each clock cycle. This can impact energy efficiency, especially in mobile and power-constrained environments.

Deeper pipelines can lead to larger code sizes, especially if additional instructions or nops are inserted to handle pipeline hazards. **Larger code sizes** can impact instruction cache efficiency and memory bandwidth.

Discussion Forum - Compiler Techniques

UNIVERSITY of York

6. How does loop unrolling contribute to performance improvement? What are the trade-offs?

Loop unrolling is an optimisation technique where multiple iterations of a loop are combined into a single iteration to reduce the overhead of loop control (e.g., incrementing counters, checking conditions).

Benefits:

Reduced loop overhead: Fewer iterations mean fewer conditional checks and branch instructions.

Better instruction-level parallelism (ILP): More instructions can be executed in parallel, reducing pipeline stalls.

Improved cache efficiency: Reduces the frequency of accessing loop variables, improving data locality.

Discussion Forum - Compiler Techniques

UNIVERSITY of York

6. How does loop unrolling contribute to performance improvement? What are the trade-offs?

Loop unrolling is an optimisation technique where multiple iterations of a loop are combined into a single iteration to reduce the overhead of loop control (e.g., incrementing counters, checking conditions).

Trade-offs:

Increased code size: If the loop body is large, unrolling can significantly increase the size of the generated code (code bloat).

Register pressure: More unrolled iterations require more registers, which can lead to register spills and degrade performance.

Diminishing returns: Unrolling too much may exceed available CPU resources, leading to no further performance gains.

Discussion Forum - Compiler Techniques

UNIVERSITY of York

7. How does instruction scheduling work, and why is it important for modern CPU architectures?

What is Instruction Scheduling?

Instruction scheduling is a compiler optimization technique that reorders instructions to minimize pipeline stalls and improve CPU efficiency. The goal is to reduce **hazards** (data, structural, and control hazards) that cause delays in instruction execution.

Why is Instruction Scheduling Important?

Modern CPUs use pipelining, where multiple instructions execute simultaneously at different stages. However, pipeline hazards can cause **stall cycles**, reducing overall performance.

Discussion Forum - Compiler Techniques

UNIVERSITY of York

7. How does instruction scheduling work, and why is it important for modern CPU architectures?

Types of Instruction Scheduling:

Static Scheduling (Compiler-Level):

- The compiler rearranges instructions at compile-time to minimize stalls.
- Works well for predictable workloads but cannot adapt to runtime changes.

Dynamic Scheduling (Hardware-Level):

- The CPU reorders instructions at runtime based on actual data dependencies.
- Used in out-of-order (OoO) execution architectures, such as Intel's **Tomasulo Algorithm**.

Discussion Forum - Compiler Techniques

7. How does instruction scheduling work, and why is it important for modern CPU architectures?

Examples of Instruction Scheduling in Action:

Load-Use Delay Mitigation: If an instruction loads data from memory, the next dependent instruction can be delayed or reordered to avoid waiting for memory access.

Branch Delay Slot Filling: A useful instruction is placed after a branch instruction to utilize the delay slot efficiently.

Challenges and Trade-offs:

Dependency Constraints: Reordering must not violate data dependencies (RAW, WAR, WAW).

Complexity: More aggressive scheduling increases compilation time and hardware complexity.

Speculative Execution Risk: If reordering is based on speculative execution, misprediction can lead to performance penalties.

Discussion Forum - Compiler Techniques

8. How do compilers optimise code for different hardware architectures (e.g., ARM vs. x86)?

Modern compilers optimize code based on the target CPU architecture, considering **instruction set differences, pipeline depth, register file size, and memory hierarchy.**

Key Optimization Techniques for Different Architectures:

Instruction Set-Specific Optimizations:

- **x86:** Uses complex instructions (CISC) with micro-op fusion for efficiency.
- **ARM:** Optimized for load-store architecture, uses **Thumb** mode for smaller code size.

Vectorization and SIMD Utilization:

- **x86 (SSE/AVX):** Uses SIMD (Single Instruction, Multiple Data) instructions for parallel data processing.
- **ARM (NEON):** Compilers optimize loops to leverage NEON for mobile and embedded systems.

Discussion Forum - Compiler Techniques

UNIVERSITY of York

8. How do compilers optimise code for different hardware architectures (e.g., ARM vs. x86)?

Pipeline and Out-of-Order Execution Awareness:

- Deep Pipelines (Intel Skylake, AMD Zen): Compilers optimize instruction scheduling to avoid pipeline stalls.
- Short Pipelines (ARM Cortex-A): Focus on reducing branch mispredictions and leveraging conditional execution.

Memory and Cache Optimisation:

- x86: Prefetching and cache-aware optimizations due to large L1-L3 caches.
- ARM: Prefers small, power-efficient cache usage, avoids unnecessary memory accesses.

Discussion Forum - Compiler Techniques

UNIVERSITY of York

8. How do compilers optimise code for different hardware architectures (e.g., ARM vs. x86)?

Register Allocation and Calling Conventions:

- x86: Uses fewer registers (e.g., x86-64 has 16 general-purpose registers).
- ARM: More registers available, enabling more aggressive register allocation and reducing memory spills.

Branch Prediction and Speculative Execution:

x86: Heavy use of speculative execution and branch prediction (e.g., Intel's **Branch Target Buffer (BTB)**).

ARM: More reliance on predicated execution and low-latency branches.

Closing Remarks and Announcement

- **Next Week (Week 8)**:

Lecture: In Week 8, our discussion will be focused on **Virtual Memory** and **Page Replacement** in Lecture 13 and Lecture 14, respectively. And we also have the 3rd tutorial lecture on **Static & Dynamic Scheduling**.

Lab: Register renaming. In previous weeks, we explored register hazards and we noted that creating delays in the pipline to solve these problems is not very efficient, as it wastes CPU clock cycles. An alternative method, register renaming, attempts to solve some of these hazards by another method, which avoids inserting stall cycles. This lab session explores that idea and a solution.

Exercises/Quizzes: Exercise 3 and Quiz 3 will be released on 17/04/2026 (Week 8 Friday). Quiz 3 will be discussed in the Lecture Review session on 23/04/2026 (Week 9 Thursday). Exercise 3 solution will be released on 24/04/2026 (Week 9 Friday) . Exercise 3 will be discussed in the Lecture Review session on 30/04/2026 (Week 10 Thursday).

Thanks !

See you After Easter!